



הפקולטה להנדסת חשמל
ע"ש אנדרו וארנה ויטרבי



הטכניון
מכון טכנולוגי לישראל

מעבדה בהנדסת חשמל
044157 א'1

חומר רקע לניסוי DEBUG ניפוי תקלות בחומרה

עבור QUARTUS SV 17

הניסוי פותח בחסות המעבדה למערכות ספרתיות מהירות 

גרסה 2.4
קיץ תשפ"ב 2022

תוכן עניינים של חומר הרקע

לניסוי DEBUG - ניפוי תקלות בחומרה

3		הקדמה	1
3		מטרות הניסוי	1.1
3		מבנה הניסוי, אופן ביצועו ודרישות מהסטודנט	1.2
4		הנתח הלוגי ב- Quartus	2
4		נתח לוגי חיצוני ופנימי	2.1
5		הכרת מושגים בסיסיים – מספר רמות המתח ותדר הדגימה	2.2
8		מושגים בסיסיים – מבנה זיכרון הדגימה ודרבון (Trigger)	2.3
9		מושגים בסיסיים – הקלטה בזיכרון סיבובי לפני ואחרי הדרבון	2.4
11		מבוא לשימוש ב- SignalTap	2.5
12		ממשק למקלדת	3
12		כללי	3.1
12		מבוא ונתונים על ממשק למקלדת ברמת החמרה	3.1.1
13		מבוא ונתונים על ממשק בין מקלדת PS/2 ו Host ברמת ה-bit	3.1.2
14		גילוי שגיאה בעזרת סיבית ה- (odd)parity	3.1.3
14		מבוא לתשדורת של המקלדת ברמת ה-byte	3.1.4
16		שימוש במקלדת מספרים	3.2
18		מימוש הממשק למקלדת	3.3
19		תכן של פילטר מעביר נמוכים	3.3.1
19		תכן יחידת ה- BITREC	3.3.2
21		תכן יחידת ה- BYTEREC	3.3.3
22		ממשק המקלדת הסופי	3.3.4
24		מערכת הניסוי לבדיקת החומרה HW_DEBUG עם מכונת RANDOM	4
25		נספח – העשרה – עריכת תוכן של זיכרון וקבועים בזמן אמת ISMCE	5
26		נספח – מימוש זיהוי ה- ENTER	6
27		נספח – טבלת קודים של כל המקשים (102)	7
28		נספח – טבלת קודים של המקשים כולל MAKE/BREAK	8

1 הקדמה

1.1 מטרות הניסוי

- ☐ תאור מממשק למקלדת
- ☐ הכרת כלי עזר בחמרה לניפוי שגיאות : Signal-TAP (נתח לוגי משובץ בחמרה)
- ☐ שימוש ב- Signal-TAP בניפוי תקלה
- ☐ שימוש ב- Signal-TAP בממשק למקלדת

1.2 מבנה הניסוי, אופן ביצועו ודרישות מהסטודנט

הניסוי מתבצע במפגש מעבדה אחד בן 4 שעות ודורש גם זמן הכנה מוקדם לפני המפגש.

לפני מפגש המעבדה יש לקרוא את כל "חומר הרקע" שמופיע בחוברת זו. בנוסף לכך יש לענות על שאלות ההכנה, להכין את הקודים ולהריץ את תרגילי ההכנה. תרגילים אלו מופיעים כולם "בדוח הכנה". בגמר ההכנה לניסוי יהיו בידך כל האלמנטים לביצוע התכן הסופי במעבדה.

דוח ההכנה שתגיש צריך לכלול: תשובות לכל השאלות, תדפיס של כל הקודים, תדפיס מסך של תוצאות הנדרשות.

במפגש המעבדה יתבצע תרגול מודרך באמצעות Quartus וכרטיס התרגול. בתכן תיצור ותבדוק חמרה שלתוכה הוכנס Logic Analyzer. בהמשך תהיה לך גם הזדמנות לשלב את כלי הדיבוג החשוב שתכיר בתכן של חמרת הממשק למקלדת PS/2 ובישום נוסף שלו.

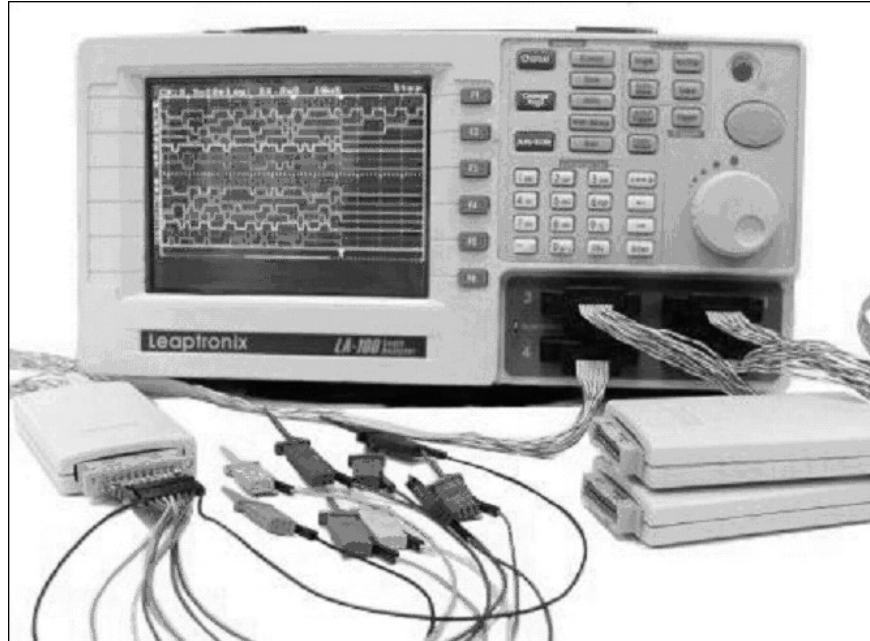
הערה חשובה:

כל הקבצים המוכנים או החצי מוכנים לניסוי זה ימצאו במודל

2 הנתח הלוגי ב- Quartus

2.1 נתח לוגי חיצוני ופנימי

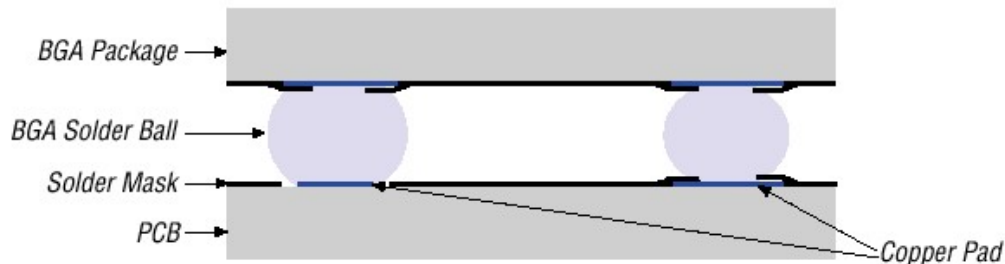
המכשיר שנקרא - Logic Analyzer הוא אחד המכשירים הבסיסיים החשובים ביותר בבדיקה של מערכות ספרתיות. מדובר במכשיר שדומה באופן פעולתו לסקופ (Oscilloscope) בעל זיכרון ומספר רב של ערוצים שמיועד ליישומים ספרתיים. נתח לוגי רגיל הוא בדרך כלל מכשור מעבדתי שמחובר באמצעות הגששים (הפרובים - Probes) שלו להדקים חיצוניים של הרכיבים או להדקי בדיקה מיוחדים של המערכת.



בדיקה של רכיבים מיתכנתים מודרניים באמצעות מכשור חיצוני כמו Logic Analyzer הולכת ונעשית מורכבת. קימות לכך כמה סיבות.

רכיבים מודרניים הולכים ונעשים מורכבים ומספר האותות הפנימיים שלהם, שעשוי להיות בעל עניין, הוא עצום! רוב האותות הפנימיים אינם מחוטים כלל מחוץ לרכיב!

בנוסף לכך הגישה להדקים של הרכיב, שאכן מחוטים החוצה, הולכת ונעשית קשה יותר הן בגלל הממדים הפיסיים הקטנים של הרכיבים והן בגלל סוגי האריזות שבהן נעשה שימוש כיום. הרכיב שבו אנו משתמשים למשל בניסוי זה (5CSXFC6D6F31C6) הוא רכיב באריזה שנקראת Fine Line BGA (השם BGA בא מהמלים Ball Grid Array). זהו רכיב שבתחתית שלו יש 896 הדקים כדורים מאוד קטנים שמולחמים למעגל המודפס!



הרגליים אינן עוברות לצד השני של המעגל המודפס (כמו באריזות DIP ישנות) ואי אפשר לחבר אליהם גששים של Logic Analyzer.

הפתרון של יצרנים של רכיבים מיתכנתים לכל הבעיות הקשות שהועלו כאן היא: לאפשר הכנסה של Logic Analyzer לתוך הרכיב. נתחים לוגיים אלו נקראים Embedded Logic Analyzers או בקצור (ELA). ה- Logic Analyzer שמוכנס לרכיבי Altera נקרא בשם SignalTap.

הנגישות הגבוהה של ה- SignalTap לאותות הפנימיים הרבים של התכן הופכת אותו לכלי דיבוג רב עוצמה!

החמרה של הנתח הלוגי ממומשת באמצעות המשאבים הלוגיים הפנימיים ומשאבי הזיכרון הפנימיים ברכיב. התכן של הנתח והשילוב שלו עם המערכת הנבדקת נעשה באופו אוטומטי באמצעות כלי הפתוח Quartus ובהתאם לדרישות המשתמש. ההורדה של התכן שכולל גם את הנתח הלוגי הפנימי נעשית באמצעות קונפיגורציה רגילה של הרכיב באמצעות שרשרת ה- JTAG.

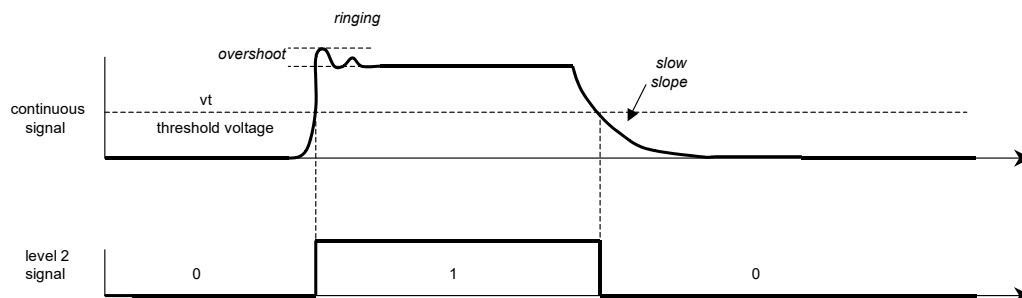
הפעלת הנתח והתקשורת עמו נעשים גם הם באמצעות שרשרת ה- JTAG. גם ההפעלה של הנתח וגם הצגת התוצאות של הנתח נעשים באמצעות תכנת ה- Quartus.

השימוש ב- SignalTap הוא נוח והוא אינו דורש רכישה של מכשור יקר!

בחלקים הבאים נכיר כמה מושגים כלליים ותכונות של מכשירי Logic Analyzer בכלל ושל ה- SignalTap בפרט.

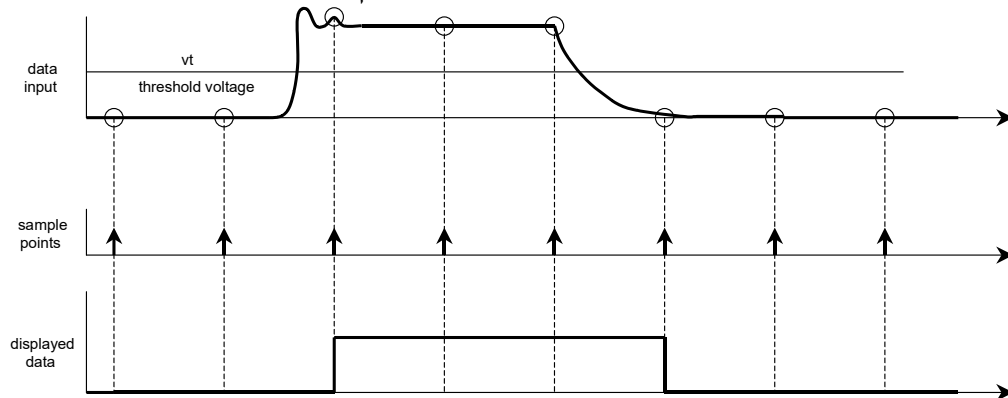
2.2 הכרת מושגים בסיסיים – מספר רמות המתח ותדר הדגימה

הנתח הלוגי (Logic Analyzer) בניגוד לסקופ (Oscilloscope) הוא מכשיר שמציג אותות ספרתיים באחת משתי רמות בלבד: '0' לוגי או '1' לוגי - וללא רמות ביניים נוספות. האיור הבא מציג דוגמה לאות כניסה כל שהוא והדרך שבה הוא מוצג באמצעות הנתח הלוגי כאות בעל שתי רמות מתח בלבד.



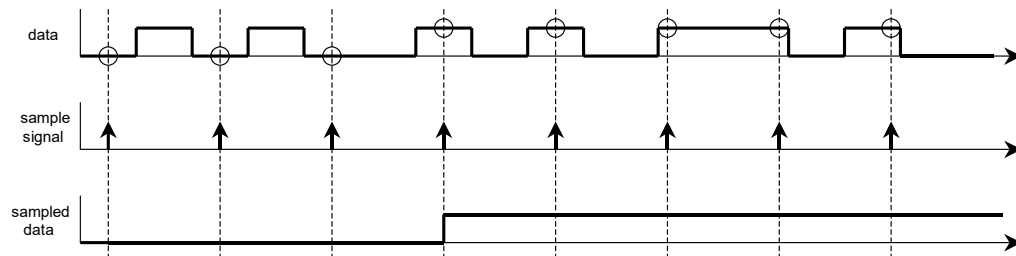
נתחים לוגיים הם מכשירים שאינם מיועדים להצגה של תופעות אנלוגיות כמו - זמני עליה (Transition Times) ושיפועים (Slew Rate or Slope) או תגובת יתר (Over-Shoot) וצלצולים (Ringing).

נתחים לוגיים הם מכשירים שדוגמים (Sample) את אות הכניסה בזמנים קצובים ומקליטים אותו (Acquisition) לזיכרון פנימי של המכשיר. האיור הבא מתאר את שעבר דגימה באמצעות הנתח הלוגי. אות הדגימה הוא האות המרכזי בדיאגרמה ונקודות הדגימה מתוארים באמצעות חצים אנכיים. רמות המתח הדגומות מתוארות באמצעות עיגולים קטנים שמצוירים על אות הכניסה.



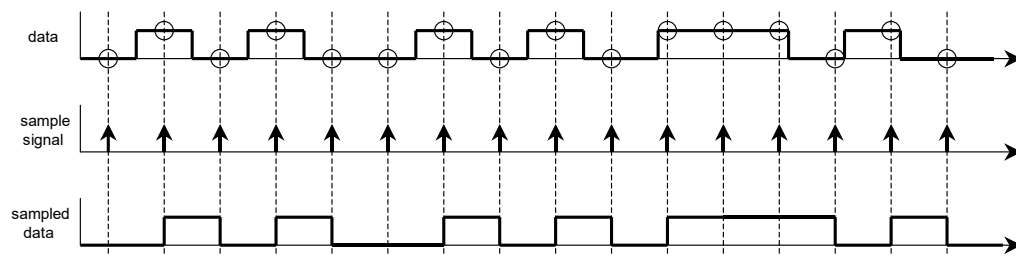
בחלק התחתון של האיור מוצג האופן שבו יוצג האות הדגום באמצעות הנתח הלוגי.

בחירת תדר הדגימה היא חשובה. בציור הבא אנו רואים בשורה הראשונה את שנקרא data שנדגם בקצב דגימה שמוצג באמצעות השורה השניה שבאיור (האות שנקרא sample signal). האות המשוחזר שמוצג על ידי ה- Logic Analyzer מוצג בשורה התחתונה שבציור (sampled data).



מכיוון שקצב הדגימה אינו גבוה מספיק, האות המשוחזר אינו דומה כלל לאות המקורי. מהו קצב הדגימה המינימלי (או זמן הדגימה המכסימלי) שמאפשר שחזור (reconstruction) של האות הנדגם? האם קיים קריטריון דומה למה שמוכר לנו בעולם האנלוגי כתדר Nyquist?

הציור הבא מדגים מקרה גבולי שבו קצב הדגימה מספיק גבוה (זמן הדגימה מספיק נמוך) בכדי לשחזר את האות din שהוצג בציור הקודם.



כפי שאפשר לראות קצב הדגימה הוא כזה, שזמן המחזור שלו מבטיח לפחות דגימה אחת בכל אזור שבו האות הנדגם נמצא ב-Low ודגימה אחת לפחות בכל אזור שבו האות הנדגם נמצא ב-High. אפשר לרשום את הקריטריון באופן הבא:

$$T_{Sample} < Min(t_{WL})$$

$$T_{Sample} < Min(t_{WH})$$

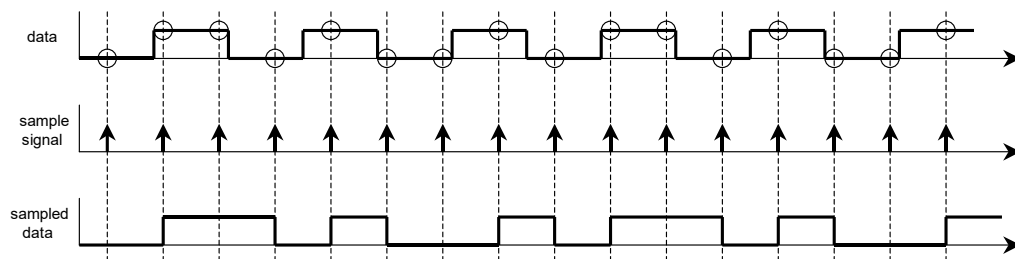
כאשר $Min(t_{WL})$ ו $Min(t_{WH})$ מתארים את רוחב הפולס המינימלי ברמות Low ו High בהתאמה של האות הנדגם. קל להראות שזהו במעין קריטריון Nyquist כפי שאנו מכירים אותו במישור התדר.

$$f_{MAX} = \frac{1}{Min(t_{WL}) + Min(t_{WH})} < \frac{1}{T_{Sample} + T_{Sample}}$$

או בקיצור :

$$2f_{MAX} < f_{Sample}$$

הקריטריון הנ"ל אמנם מספיק בכדי שנוכל לשחזר את האות מבלי "לפספס" שינויים כלשהם באות, אך בדרך כלל נעדיף לדגום אותות בקצב גבוה יותר (בזמן מחזור קצר יותר) בכדי שהם יראו טוב יותר. הדוגמה הבאה מדגימה למה הכוונה.



קצב הדגימה הוא אמנם קצת גבוה יותר מהמינימום הנדרש בכדי לקיים את הקריטריון הנ"ל. למרות זאת, אם מתבוננים באות המשוחזר נראה שהאות המקורי אינו אות בעל קצב אחיד. כמובן שאין הדבר כך. האות המקורי בדוגמה הזאת הוא אות מחזורי לחלוטין.

בכדי למנוע עיוותים מסוג זה במישור הזמן, בדרך כלל משתמשים בתדר דגימה גבוה יותר (זמן דגימה נמוך יותר). כמובן שככל שתדר הדגימה גבוה יותר מידות הזמן של האות המשוחזר מדויקות יותר. מצד שני אם משתמשים בתדר דגימה גבוה מאוד (זמן דגימה נמוך מאוד) מבזבזים משאבי זיכרון יקרים של ה - Logic Analyzer.

בדרך כלל כאשר משתמשים ב - Logic Analyzer למטרות דיבוג כללי (ולא למטרת מדידות זמן מדויקות) נוהגים להשתמש בזמני דגימה שהם קטנים לפחות פי עשר מזמן הדגימה שהוצג בחלק הקודם :

$$T_{Sample} < \frac{Min(t_{WL})}{10}$$

$$T_{Sample} < \frac{Min(t_{WH})}{10}$$

כאשר רוצים לבצע מדידות זמן מדויקות יותר יש להשתמש בזמני דגימה נמוכים יותר (תלוי כמובן באחוז השגיאה במדידה שאותו אנו מוכנים לסבול).

2.3 מושגים בסיסיים – מבנה זיכרון הדגימה ודברון (Trigger)

נתחים לוגיים הם מכשירים שדוגמים בדרך כלל יותר מסיבית אחת בבת אחת (למשל 32 סיביות). מספר הערוצים (Number of Channels) או רוחב המידע (Data width) של ה-SignalTap יכול לנוע בין סיבית בודדת ל-1024 סיביות.

מספר הדגימות שהמכשיר מסוגל להקליט (Number Of Samples) או עומק הדגימה (Sample Depth) של ה-SignalTap יכול לנוע בין 64 ל-128 K. להלן רשימת כל האפשרויות: 64, 128K, 64K, 32K, 16K, 8K, 4K, 2K, 1K, 512, 256, 128.

קצב הדגימה נקבע בדרך כלל באמצעות שעון הדגימה (Trigger Clock). לעתים קרובות זהו פשוט אות השעון הראשי של המערכת, שהוא גם בדרך כלל האות בעל התדר הגבוה ביותר במערכת. כאשר מוכנים לוותר על הדיוק ורוצים לחסוך במשאבי הזיכרון של המערכת משתמשים באות דגימה בעל תדר נמוך יותר מתדר השעון של המערכת.

לדוגמה, כיצד לחשב מה עומק הזכרון שצריכים להקצות ב-Signal Tap, כדי לקלוט קוד שלם של מקש (שמכיל 11 סיביות)? נשתמש בכלל:

$$\text{bits} * \text{time} * \text{frequency} = \text{memory}$$

על פי הכלל הנ"ל נבצע את החישוב באופן הבא:

- זמן מחזור שעון אחד של PS2_CLK (זמן של סיבית אחת): 80usec
- מספר הסיביות בקוד של מקש אחד: 11 (עוברת סיבית אחת בכל פולס שעון)
- משך הזמן של קוד שלם של מקש: $80 \times 11 = 880 \text{usec}$
- התדר של שעון הדגימה (השעון בלוח התרגול DE10): 50MHz
- מספר הדגימות לקליטת קוד שלם של מקש: $880 \text{us} * 50 \text{ MHz} = 44 \text{k}$

מסקנה: בנתח הלוגי יש לבחור עומק זיכרון הקטן ביותר האפשרי שיכול להכיל 44k: לכן נבחר עומק זכרון של 64K ב-Signal Tap.

אות ה-Trigger (אות הדברון) הוא האות שמורה לנתח הלוגי לעצור את הקלטת האותות הנבחרים לזיכרון ה-RAM. אות ה-Trigger יכול להיות אות חיצוני שמגיע למערכת (Trigger In Signal) או שהוא יכול להיווצר באופן פנימי מהאותות הנדגמים עצמם.

אות ה-Trigger הפנימי נוצר בדרך כלל באמצעות בדיקה של צירוף כל שהוא של הסיביות הנדגמות. בדרך כלל בודקים שכל אחת מהסיביות הנדגמות מקיימת שילוב כל שהוא של אחד מהתנאים האפשריים הבאים:

Don't Care	
Rising Edge	
Falling Edge	
Either Edge	
High	
Low	

נמחיש את מה שנאמר כאן בדוגמה. נניח שאנו קובעים תנאי דברון (Trigger Condition) הבא על ששת אותות הכניסה - D[5..0]:

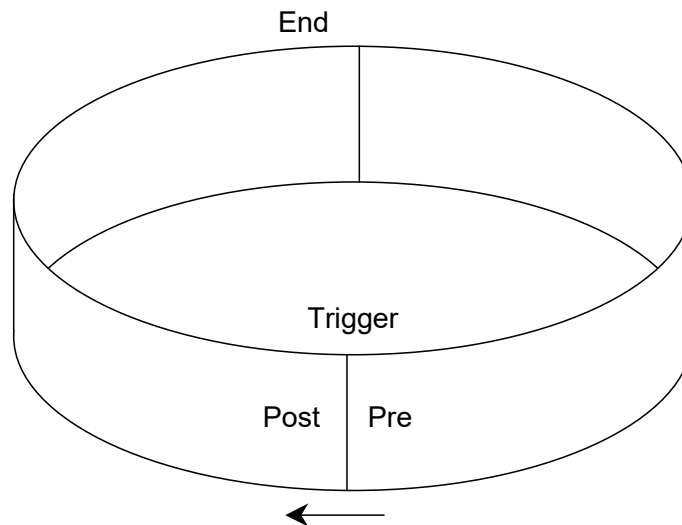
- ☐ עליה (Rising Edge) באות D0
- ☐ האותות D1 ו D2 ימצאו ה - '0' לוגי (Low)
- ☐ האות D3 ימצא ב - '1' לוגי (High)
- ☐ האותות האחרים (D4 ו D5) אינם חשובים (Don't Care)

בכל פעם שהתנאי הנ"ל יתקיים המערכת תעצור את הקליטה והרישום של הדגימות לזיכרון של הנתח הלוגי.

ניתן ליצור תנאי Trigger מורכבים יותר באמצעות כמה מאורעות שאינם מתבצעים בו זמנית. הנתח ממתין לביצוע התנאי הראשון ולאחר מכן הוא ממתין לביצוע התנאי השני וכו... עד לביצוע כל התנאים. לכל תנאי כזה קוראים Trigger Level. ב - SignalTap ניתן ליצור עד עשרה Trigger Levels.

2.4 מושגים בסיסיים – הקלטה בזיכרון סיבובי לפני ואחרי הדרבון

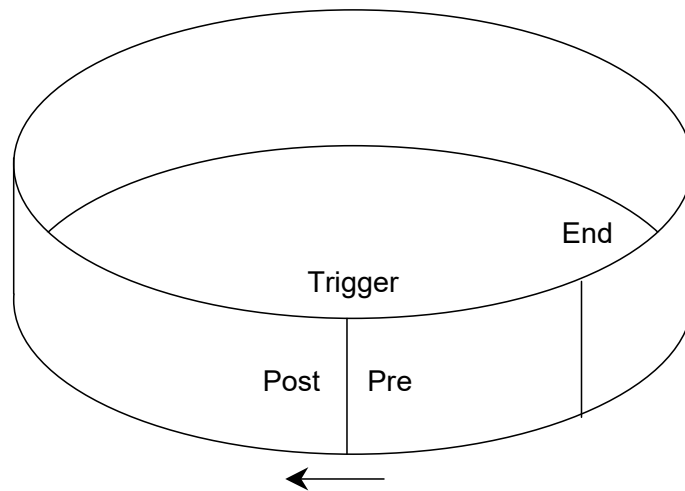
אפשרות חשובה שקיימת במכשירי Logic Analyzer היא להציג את מה שקרה לפני ה - Trigger. רישום של מאורעות שקרו לפני ה - Trigger מתאפשר באמצעות ההקלטה (Acquisition) שנעשית באופן מתמיד גם לפני ה - Trigger לתוך זיכרון סיבובי (Circular Buffer), כאשר הדגימות החדשות "דורסות" ללא הרף דגימות ישנות. מרגע שמתקבל Trigger, ממשיכים לאחסן דגימות רק בחלק מהזיכרון הסיבובי. הציור הבא מדגים מקרה שבו ממלאים חצי מהזיכרון בדגימות חדשות שנעשות לאחר ה - Trigger.



בגמר פעולה זו, חצי מהזיכרון (בצד ימין באיור) מכיל דגימות שנעשו לפני ה - Trigger וחצי מהזיכרון (בצד שמאל באיור) מכיל דגימות שנעשו לאחר ה - Trigger. לצורת עבודה זו קוראים:

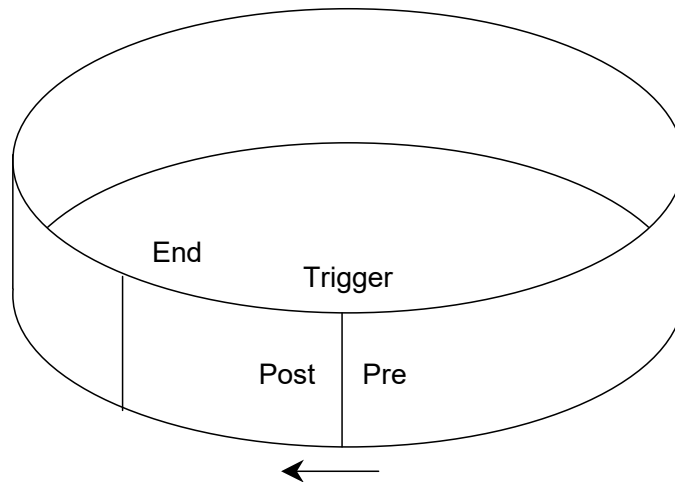
50 % Pre-Trigger / 50% Post-Trigger

הציור הבא מדגים מקרה שבו ממשיכים למלא יותר ממחצית מהזיכרון בדגימות חדשות שנעשות לאחר ה - Trigger.



בצורת עבודה זו רוב הזיכרון יהיה מוקצה להקלטות שנעשו אחרי ה - Trigger.

הציור הבא מדגים מקרה שבו ממשיכים למלא פחות ממחצית מהזיכרון בדגימות חדשות שנעשות לאחר ה - Trigger.

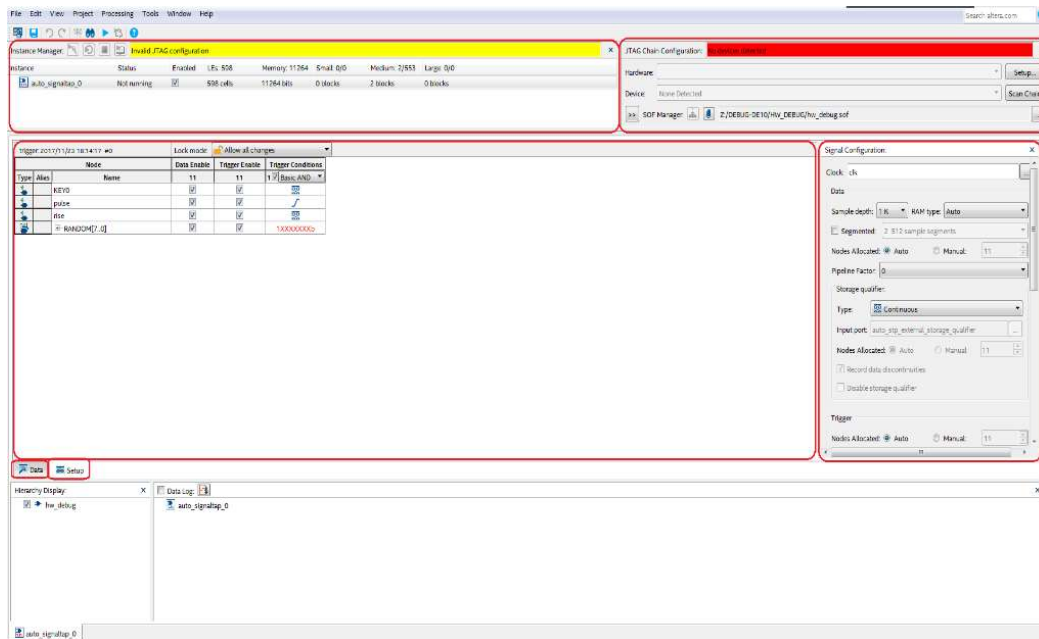


בצורת עבודה זו רוב הזיכרון יהיה מוקצה להקלטות שנעשו לפני ה - Trigger. ב - SignalTap קיימים שלושה מצב עבודה (Trigger Position) שדומים למה שתואר בציורים הנ"ל:

- ☐ מצב - Pre Trigger position (12%Pre trigger / 88% Post-Trigger)
- ☐ מצב - Center Trigger Position (50%Pre trigger / 50% Post-Trigger)
- ☐ מצב - Post Trigger position (88%Pre trigger / 12% Post-Trigger)

2.5 מבוא לשימוש ב - SignalTap

הקונפיגורציה של ה - SignalTap (ההדקים הנבדקים, אות השעון שמשמש לדגימה, מיקום ה - Trigger, מספר הדגימות ועוד ...) נעשים כולם באמצעות Quartus. מידע הזה מאוחסן בקובץ בעל סיומת *.stp.



להלן פירוט קצר על כל אחד מהאזורים המסומנים :

- צד ימין עליון :** הגדרת הכרטיס והפעלת הצורב.
- צד ימין אמצעי :** הגדרת השעון, עומק הזכרון ואופן הדגימה.
- צד שמאל עליון :** אפשרויות ההפעלה ומעקב הביצוע.
- צד שמאל אמצעי :** הגדרת האותות להקלטה ודברון.

לאחר ביצוע קונפיגורציה יש לקמפל את ה - SignalTap כך שהוא יהיה חלק אינטגרלי של התכן.



את קובץ הצריבה של התכן שכולל את ה - SignalTap מורידים לרכיב באמצעות הדקי ה - JTAG. (USB)



ההפעלה של ה - SignalTap, נעשית באמצעות Quartus, על ידי התפריטים :

Tools -> SignalTAP II Logic Analyzer

התקשורת עם ה - SignalTap נעשית באמצעות ההדקים של מערכת ה - JTAG. (USB)

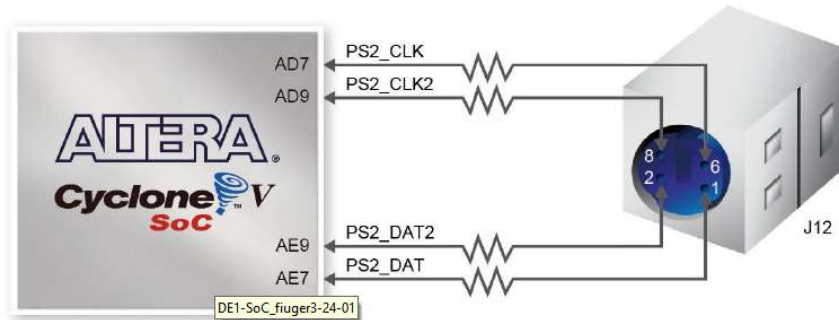
בניסוי שתבצע במעבדה תכיר באופן מפורט ומעשי את השלבים שאותם יש לבצע על מנת שתוכל להשתמש ב - SignalTap. פרוט על ה - SignalTap - תוכל למצוא ב Quartus Cook Book

3 ממשק למקלדת

3.1 כללי

3.1.1 מבוא ונתונים על ממשק למקלדת ברמת החמרה

בניסוי ניצור ממשק למקלדת PS/2. המקלדת מתחברת בדרך כלל למחשב (Host) או במקרה שלנו ללוח התרגול (DE10) באמצעות כבל הכולל ארבעה חוטים:



האות PS2_CLK מופק במקלדת ומשמש ככניסה של ה-Host. האות PS2_DAT כפי שהוא מוגדר בפרוטוקול PS/2 הוא אות דו כיווני שיכול להיות מופק במקלדת אך יכול להיות מופק גם על ידי ה-Host. בניסוי זה נמש על גבי לוח התרגול Host שמסוגל רק לקבל מידע מהמקלדת ואינו שולח חזרה מידע למקלדת. לכן במקרה שלנו האות PS2_DAT הוא אות שמופק במקלדת ומשמש ככניסה ללוח התרגול בלבד.

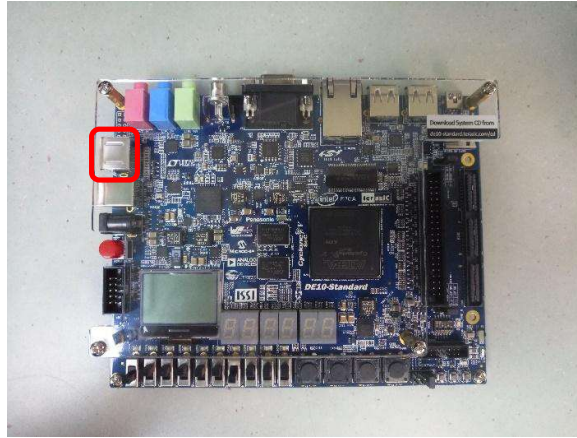
המחבר שבו נעשה שימוש הוא מחבר בעל שישה הדקים (6 pin mini-DIN PS/2 style connector) שנראה כך:



להלן תפקידי ההדקים השונים במחבר:

Pin on connector	Function	
1	PS2 DAT	מידע של המקלדת
2	Not Connected	
3	GND	
4	VCC	
5	PS2 CLK	שעון של המקלדת
6	Not Connected	

מחבר זה נמצא בצדו השמאלי העליון של לוח התרגול DE10: ומגיע למחבר בחזית



להלן הקצאת ההדקים של הכניסות הנ"ל מנקודת מבט של רכיב ה- Cyclone שעל לוח התרגול.

Signal Name	Name on script	Pin on DE-10
PS2_CLK	PS2_CLK	PIN_AB25
PS2_DAT	PS2_DAT	PIN_AA25

https://en.m.wikipedia.org/wiki/PS/2_port ראה גם

3.1.2 מבוא ונתונים על ממשק בין מקלדת PS/2 ו Host ברמת ה-bit

פרוטוקול PS/2 שימושי במקלדות של מחשבי PC.

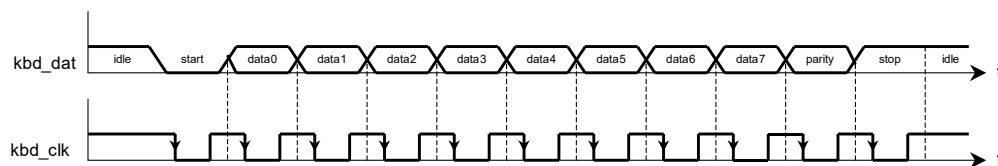
פרוטוקול התשדורת PS/2 הוא פרוטוקול סינכרוני וטורי כלומר מתבצעת העברה של המידע (באמצעות האות PS2_DAT) ביט אחרי ביט ובאופן מתואם לאות שעון (PS2_CLK).

במצב מנוחה, שבו אין תשדורת (Idle או Inactive), המצב הלוגי של PS2_DAT הוא 1' לוגי. התשדורת מתחילה בסיבית התחלה (Start Bit) שהיא תמיד 0' לוגי. לאחר מכן משודר רצף של שמונה סיביות החל מסיבית ה- LSB עד לסיבית ה- MSB.

לאחר מכן משודרת סיבית הזוגיות (Parity), שצריכה ליצור מצב שבו מספר האחדים בתשדורת (כולל סיבית ה- Parity עצמה) הוא תמיד אי-זוגי (Odd Parity).

לאחר מכן משודרת סיבית הסיום (Stop Bit), שהיא תמיד 1' לוגי. בגמר פעולות אלו המקלדת חוזרת למצב מנוחה, עד לתשדורת הבאה.

האיור הבא מציג דיאגרמת זמנים שמתארת את צורת התשדורת של המידע יחד עם אות השעון של המקלדת.



שים לב שבדיאגרמות הזמנים הנ"ל החלפת המידע נעשית תמיד בזמן שאות השעון הוא 1 לוגי. בזמן שאות השעון של המקלדת הוא 0 לוגי, המידע תמיד יציב וניתן לדגום אותו בזמן זה. בתכן שבניסוי זה נדגום את המידע בעליה באות השעון של המקלדת.

להלן דוגמה להקלטה של אותות ממקלדת אמיתית באמצעות ה-SignalTAP שבו נעסוק בחלק הבא של הניסוי.



3.1.3 גילוי שגיאה בעזרת סיבית ה- (odd)parity

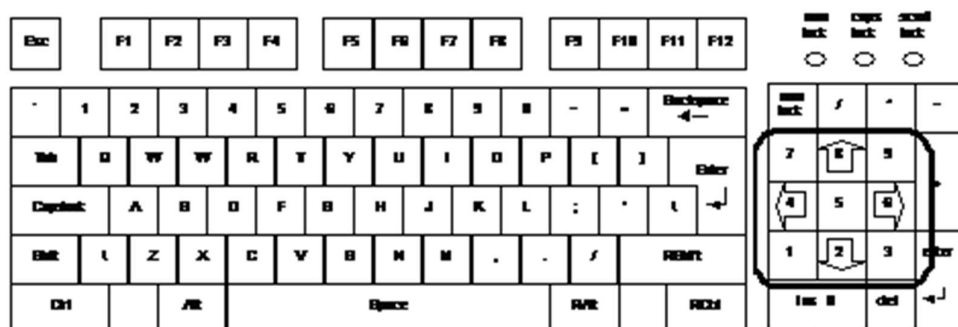
התחנה המשדרת (למשל keyboard) מחשבת את סיבית הזוגיות (PB) באופן הבא:
 $PB_{transmit} = NOT (D0 \text{ xor } D1 \text{ xor } D2 \dots \text{ xor } D7)$

התחנה הקולטת בודקת את סיבית הזוגיות (PB) באופן הבא:
 $PB_{receive} = NOT (D0 \text{ xor } D1 \text{ xor } D2 \dots \text{ xor } D7)$
 If (PB_{receive} = PB_{transmit}) then O.K.
 else ERROR

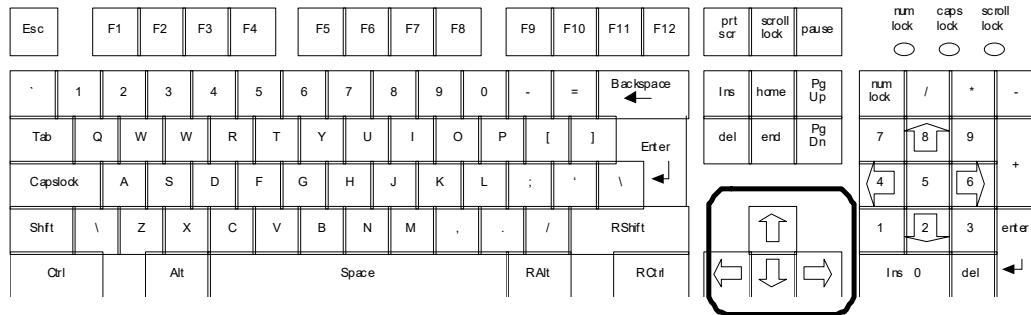
האלגוריתם הנ"ל מגלה שגיאה רק אם חל שינוי במספר אי-זוגי של סיביות ולא מגלה שגיאה אם חל שינוי במספר זוגי של סיביות. בכל מקרה, אין הוא יודע לתקן את השגיאה. קיימים אלגוריתמים מורכבים יותר שבהם מוסיפים סיביות נוספות למידע הנבדק ובעזרתן ניתן לגלות ולתקן מספר גדול יותר של שגיאות.
 ביישום שלנו, הזוגיות נבדקת על 9 ביטים של המילה המועברת, 8 ביטים של קוד המקש וביט הזוגיות. לא נכללים בבדיקה הביטים הידועים של התחלה (Start) וסיום (Stop) המילה המועברת.

3.1.4 מבוא לתשדורת של המקלדת ברמת ה-byte

ברמת ה-byte, צריכים לקלוט את ה-bits של השידור ולפרש מצירופים אלו, מהו המקש שנלחץ או נעזב. כדי להדגים זאת, נשתמש למשל בדוגמה של מקשי החצים (Arrows) במקלדת. במקלדות ישנות שהיו במחשבי PC ו XT מקשי החצים נמצאו אך ורק בחלקה הימני של המקלדת.



במקלדות מודרניות יותר, שהוכנסו לשימוש בתקופת מחשבי ה-AT, הוספו למקלדת גם מקשי חצים נוספים בחלק התחתון של המקלדת משמאל לקבוצת המקשים הקודמת.



הקשה למשל על מקש החץ שמורה כלפי מטה (Down Arrow), שנמצא באזור הימני שבמקלדת, מפיקה את ה - Byte הבא (בשיטת ייצוג הקסדצימלית):

72

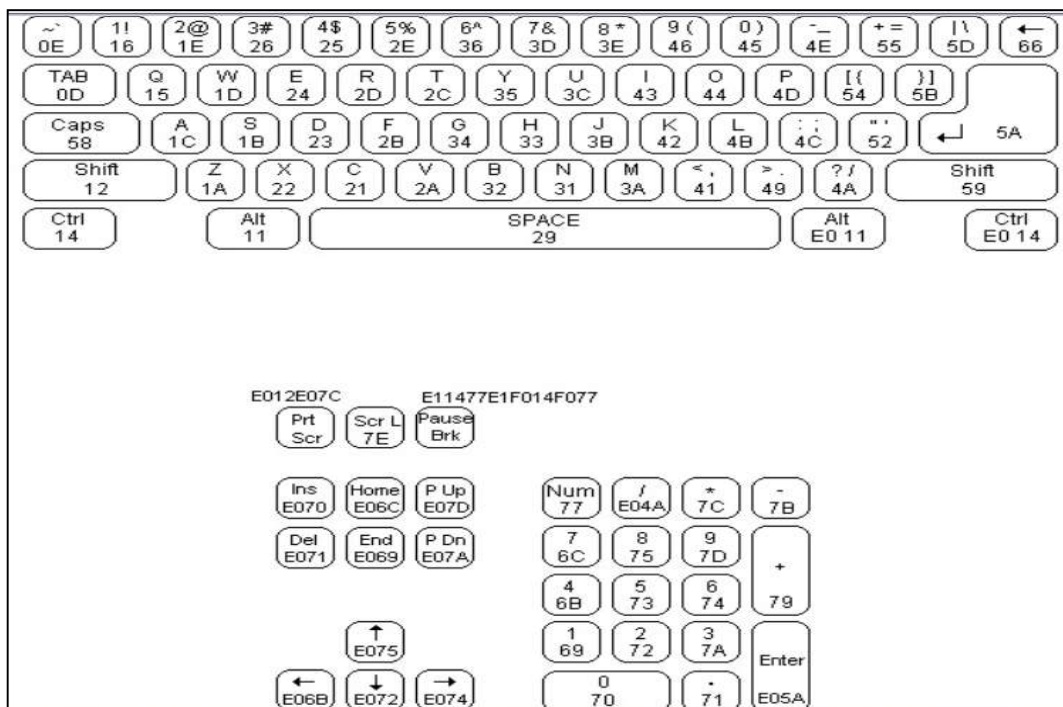
עזיבה של המקש גורמת לייצור byte נוסף שערכו הוא F0 ושמלווה ב - byte נוסף של המקש (72), כלומר מתווספים ל - byte המקורי הנ"ל של המקש שני bytes נוספים.

72 F0-72

ה - byte שערכו הוא F0, משמש תמיד כקידומת של שחרור המקש שנלחץ והוא נקרא בשם Release byte. לצירוף F0-72 בסוף התשדורת הנ"ל קוראים לפעמים גם בשם Break-Code (קוד עזיבה). ל - byte הראשון בתשדורת הנ"ל קוראים לפעמים גם בשם Make-Code (קוד לחיצה).

ערכי קוד של מקשים כדוגמת הערך 72, שהוא הערך של מקש ה - Down Arrow, יכולים לנוע בין הערכים 1 עד 84 עבור המקשים השונים של המקלדת. למשל הערך 75 הוא הערך של המקש Up-Arrow. לשאר הערכים (כמו F0 או E0 וערכים אחרים) יש משמעויות שונות.

להלן נתונים המקשים השונים והקודים שלהם במקלדת בה תשתמש במעבדה.



לחיצה ארוכה על מקש במקלדת, יוצרת פעולת Auto-Repeat, כלומר הקוד של המקש שנלחץ משוכפל מספר פעמים. לדוגמה לחיצה ארוכה על מקש Down Arrow, תגרום לקבלת רצף של כמה bytes שערכם הוא 72. להלן דוגמה לרצף שנוצר מלחיצה ארוכה של מקש זה ועזיבתו.

72 72 72 72 .. F0-72

כפי שאפשר לראות, גם בסופה של לחיצה ארוכה זו נוצר קוד שחרור עם הערכים F0-72.

גם מקש ה- Down Arrow המודרני יותר (שנמצא כאמור קודם בחלק התחתון של המקלדת משמאל לקבוצת המקשים הני"ל), מפיק תוצאה מספרית 72, אך רצף ה- bytes שנוצר קצת שונה. להלן תוצאת לחיצה ארוכה על מקש זה:

E0-72 E0-72 E0-72 .. E0-F0-72

ה- byte שערכו הוא E0 הוא הקידומת של מקשים שמיצרים Extended-Codes (קוד מורחב). שים לב שהקידומת E0 מופיעה לפני כל הפעלה חוזרת של המקש (Auto Repeat) וגם לפני העזיבה (Release) של המקש.

למשל, בהפעלת מקש ה- down arrow אחת מ-4 הסדרות הבאות אפשריות:

Extended code	Normal code	לחיצה/שחרור
E0H, 72H	72H	לחיצת מקש
E0H, F0H, 72H	F0H, 72H	שחרור מקש

הקודים המורחבים אפשרו במשך השנים להוסיף למקלדות מקשים שונים. הוספה כזו נעשתה למשל במעבר ממקלדות XT למקלדות AT וכן כיום במקלדות מודרניות יותר שתומכות בפעולות שונות של Windows. כפי שראינו כבר קודם בדוגמה של מקש ה- Down-Arrow, במקלדות מודרניות, קיימים מקשים בעלי תפקידים זהים שנמצאים ב**שני** מקומות שונים במקלדת. הקוד הבסיסי של רוב המקשים הכפולים הללו זהה אך אחד מהם הוא רגיל והשני הוא מורחב. דוגמאות אחרות פרט לארבעת סוגי מקשי החצים הם למשל מקשי הדפדוף (PgDn ו PgUp), מקשי ה- Enter, Ctrl, Alt, Delete, Insert, End, Home ו השימוש בקודים זהים (רגילים ומורחבים למקשים כפולים) עזר כמובן ליצרנים של ציוד במעבר ממקלדות XT למקלדות AT.

<< הממשק במעבדה הוא של 9 BIT אם הביט התשיעי =1 זהו מקש EXTENDED (E0) >>

3.2 שמוש במקלדת מספרים

עקב השינויים שחלו לאחרונה על אופן הלמידה בטכניון, המקלדת השלמה הוחלפה בניסוי זה על ידי מקלדת מספרים, בעלת 18 מקשים בלבד: כל 17 מקשי ה- "Numbers pad" ועוד המקש של "Back space", כמתואר בשתי התמונות להלן.



במקלדת המספרים למקשים יש אותם קודים כמו לאזור המספרים במקלדת הגדולה (ראה קודים נתונים בסעיף 3.1.4). במקלדת זו ישנם רק שני מקשים בעלי קוד מסוג Extended:

- המקש: Enter – הקוד: E05A
- המקש: / – הקוד: E04A

מקלדת המספרים מתחברת לכרטיס DE10 באותו אופן כמו המקלדת המלאה, דרך כניסת ה-PS2, והתקשורת עם הכרטיס היא כמו במקלדת המלאה, דרך שני האותות PS2_CLK ו-PS2_DAT.

DEBUG – הקודים של לוח המקשים

מקשי מקלדת גדולה והקודים שלהם

~` 0E	1! 16	2@ 1E	3# 26	4\$ 25	5% 2E	6^ 36	7& 3D	8* 3E	9(46	0) 45	-= 4E	+= 55	[\ 5D	← 66	
TAB 0D	Q 15	W 1D	E 24	R 2D	T 2C	Y 35	U 3C	I 43	O 44	P 4D	[{ 54	}] 5B			
Caps 58	A 1C	S 1B	D 23	F 2B	G 34	H 33	J 3B	K 42	L 4B	;; 4C	"" 52	↵ 5A			
Shift 12	Z 1A	X 22	C 21	V 2A	B 32	N 31	M 3A	<. 41	>. 49	?/ 4A	Shift 59				
Ctrl 14	Alt 11	SPACE 29										Alt E0 11	Ctrl E0 14		

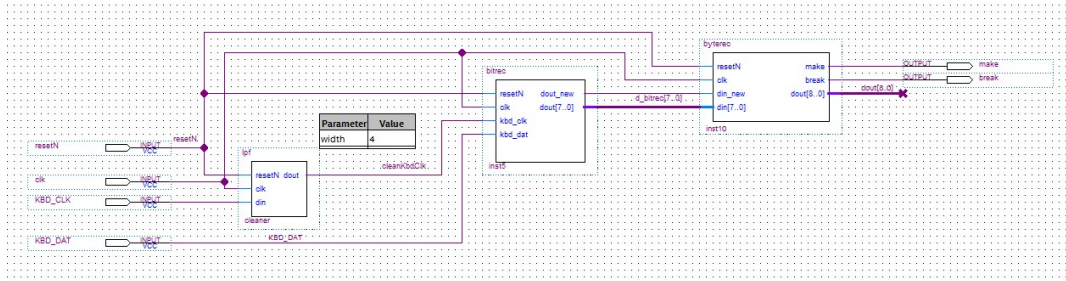
קודים עם מקלדת מספרים

E012E07C			E11477E1F014F077		
Prt Scr E070	Scr L 7E	Pause Brk E07D			
Ins E070	Home E06C	P Up E07D			
Del E071	End E069	P Dn E07A			
↑ E075					
← E06B					
↓ E072					
→ E074					

Num 77	/ E04A	* 7C	- 7B
7 6C	8 75	9 7D	+
4 6B	5 73	6 74	79
1 69	2 72	3 7A	Enter E05A
0 70	.71		

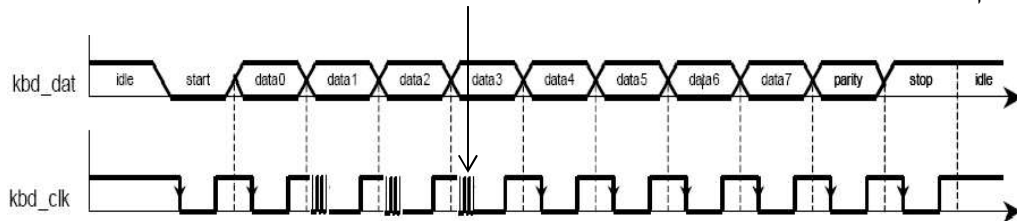
3.3 מימוש הממשק למקלדת

ניתן לממש ממשק חומרה למקלדת באופנים שונים. התכן הסינכרוני שבו בחרנו יאפשר תמיכה בכל מקשי המקלדת, יהיה מאוד נוח לשימוש, יהיה גמיש וניתן להרחבה ליישומים רבים ויהיה גם קל להבנה. נפרק את התכן של ממשק המקלדת לשלושת החלקים הבאים.



LPF - הבלוק השמאלי שנקרא LPF (מסנן מעביר נמוכים), הוא פילטר שמסנן רעשים. אותות המידע והשעון (PS2_CLK ו PS2_DAT) שמגיעים ממקלדות מסוגים שונים, עלולים להיות לפעמים רועשים. מדובר בעלויות וירידות בלתי נקיות שעלולים להיווצר כתוצאה מ Hazards – בחמרת היציאה של המקלדת או כתוצאה מהחזרות בחוט המקשר בין המקלדת ולוח התרגול. למרות שתופעות הללו לא תמיד קורות בכל מקלדת ולמרות שלעיתים קרובות אפשר ליצור תכן ללא יחידת LPF, מומלץ בכל זאת שלא לקחת סיכון.

היות ובתכן שלנו אנו עומדים להסתמך על העלויות והירידות באות השעון של המקלדת (PS2_CLK), חשוב לסנן עלויות וירידות בלתי רצויות שנגרמות באות הרועש הזה, כמו בדוגמה להלן.



ה – LPF מעביר ליציאה שלו אך ורק אותות שהם עקביים לפחות מספר מסוים של מחזורי שעון. אין צורך לסנן באופן דומה את אות המידע של המקלדת (PS2_DAT), היות ואות זה ידגם בתכן שלנו באזורים שבהם הוא אינו משתנה (מיד לאחר הירידה של האות PS2_CLK).

BITREC - היחידה שמוצגת במרכז האזור הנ"ל, שנקראת BITREC (מקלט ברמת ה – Bit), היא יחידה שקולטת את המידע הטורי שמגיע מהמקלדת לכניסה PS2_DAT. יחידה זו דוגמת את המידע הטורי בעלית השעון הנקי של המקלדת (האות שמגיע מיחידת ה – LPF). יחידה זו גם ממירה את המידע הטורי למידע מקבילי, בודקת את תקינותו (Start Bit, Parity, Stop Bit) ורק אם המידע תקין, היא מעבירה אותו ליציאה dout. יחידה זו מייצרת גם אות חייווי שנקרא dout_new שמסמן הגעה של מידע חדש ליציאה dout.

BYTEREC - היחידה הימנית שבאזור ושנקראת BYTEREC (מקלט ברמת ה – Byte), מקבלת סדרה של כמה Bytes במקביל מהיחידה הקודמת. הסדרה שמתקבלת מפיקה ביציאה dout קוד שמראה איזה מקש נלחץ (Scan Code). כל הכנסה של כל byte ליחידה זו נעשית כאשר הכניסה din_new נמצאת ב – 1' לוגי. שים לב שהקוד המופק ביציאה dout הוא ברוחב של 9 סיביות. שמונת הסיביות הנמוכות מייצרות את ה – Scan-Code של המקש. הסיבית הגבוהה ביותר (MSB)

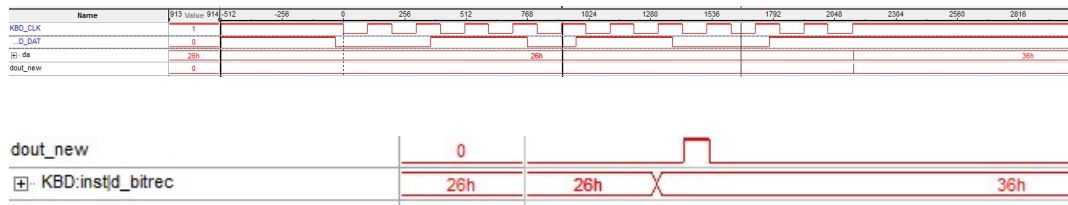
מראה, האם מדובר במקשים רגילים או במקשים מורחבים (Extended). יחידה זו גם מפיקה חיווי האם המקש שהקוד שלו מופיע ביציאה **נלחץ** כרגע (האות make) או **נעזב** כרגע (האות break).

3.3.1 תכן של פילטר מעביר נמוכים

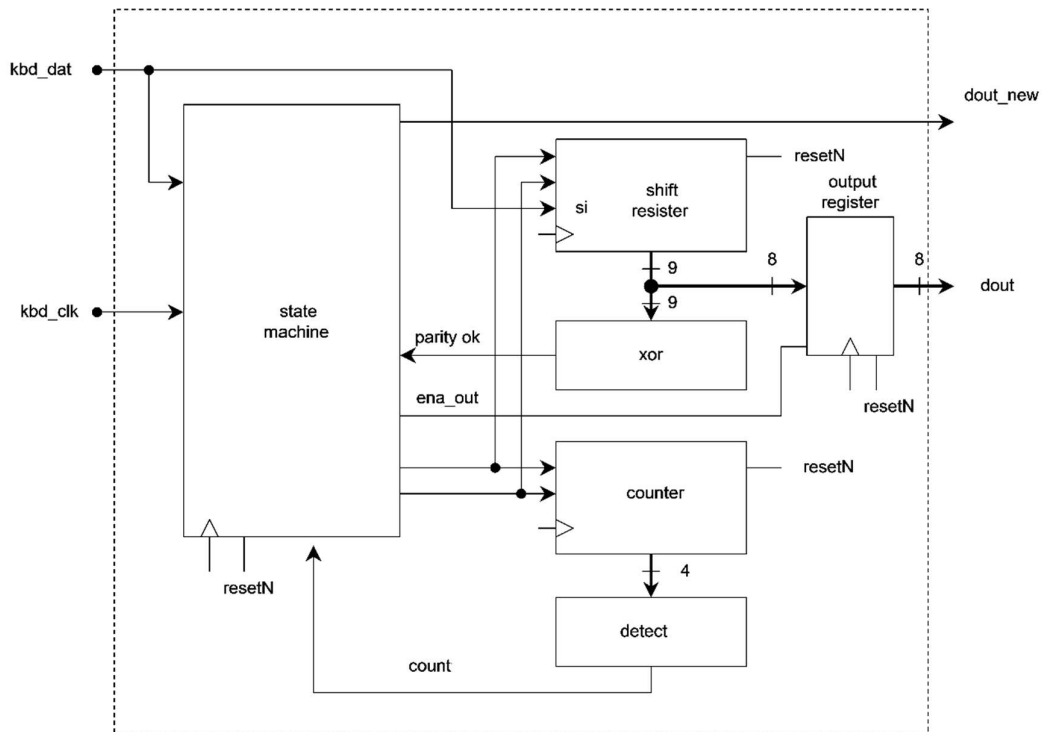
כאמור היחידה LPF מסננת את הרעשים מהקו של שעון המקלדת (PS2_CLK). ניתן לממש מסנן מסוג זה בדרכים שונות, ביישום שלנו ממומש מסנן של 4 סיביות והוא נתון בפרויקט שלך.

3.3.2 BITREC תכן יחידת ה - BITREC

תפקידה של היחידה BITREC שמטפלת בתשדורת הטורית הוא להפיק מהמידע הטורי שמגיע לכניסות PS2_CLK ו- PS2_DAT מידע מקבילי ביציאה dout, יחד עם יציאת חיווי שפעילה למשך מחזור שעון אחד ושנקראת dout_new. דיאגרמת הזמנים הבאה מתארת אותות אלו אחד ביחס לשני.

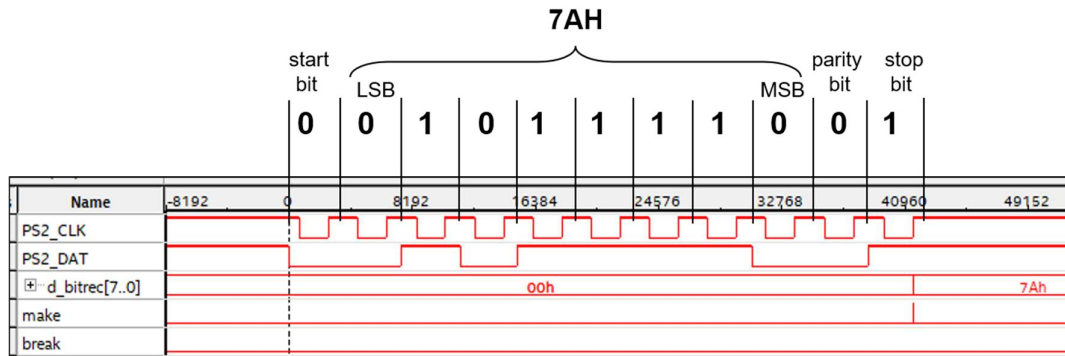


שים לב שהאות dout_new פעיל במשך מחזור שעון אחד **לאחר** שהמידע dout עודכן במידע חדש. האיור הבא מתאר את המבנה הפנימי של היחידה ברמת תהליכים עקרוניים (צירופיים וסינכרוניים).



המערכת ממומשת כמכונת מצבים אחת. פונקציונלית היא כוללת רגיסטר הזזה (shift register) ימינה ברוחב X סיביות, שתפקידו הוא לאסוף מאות הכניסה PS2_DAT את סיביות המידע וסיבית ה - parity. הסיבית הנמוכה (LSB) של רגיסטר ההזזה מאחסנת בגמר קליטת המידע הטורי את הסיבית הראשונה של התשדורת. הסיבית הגבוהה (MSB) של רגיסטר ההזזה מאחסנת בגמר קליטת המידע הטורי את סיבית ה - Parity של התשדורת כמודגם באיור הבא.

ממשק המקלדת שולח באופן טורי את המידע על שני חוטים PS2_DAT ו- PS2_CLK. המידע נדגם בעלית השעון PS2_CLK. הסיבית הראשונה היא Start, אחריה מגיעות שמונה סיביות של קוד המקש, סיבית ה- Parity, וסיבית ה- Stop שמסיימת את המילה. סך הכל 11 סיביות עוברות בתקשורת הטורית לכל מילה.



תשע היציאות במקביל של רגיסטר ההזזה מזינות מערכת צירופית (שבנויה משער XOR) שקובעת האם ה - parity של התשדורת תקין. האות שמופק מיחידה צירופית זו נקרא parity_ok והוא מוזן חזרה למכונת המצבים.

שמונה הסיביות הנמוכות של רגיסטר ההזזה (8 מתוך 9) מוזנות לרגיסטר היציאה. בניגוד לאופן פעולתו של רגיסטר ההזזה שהתוכן שלו משתנה במהלך התשדורת, תפקידו של רגיסטר היציאה הוא לאחסן את התוצאה הסופית הנקיה בלבד. רגיסטר זה הוא חלק ממכונת המצבים.

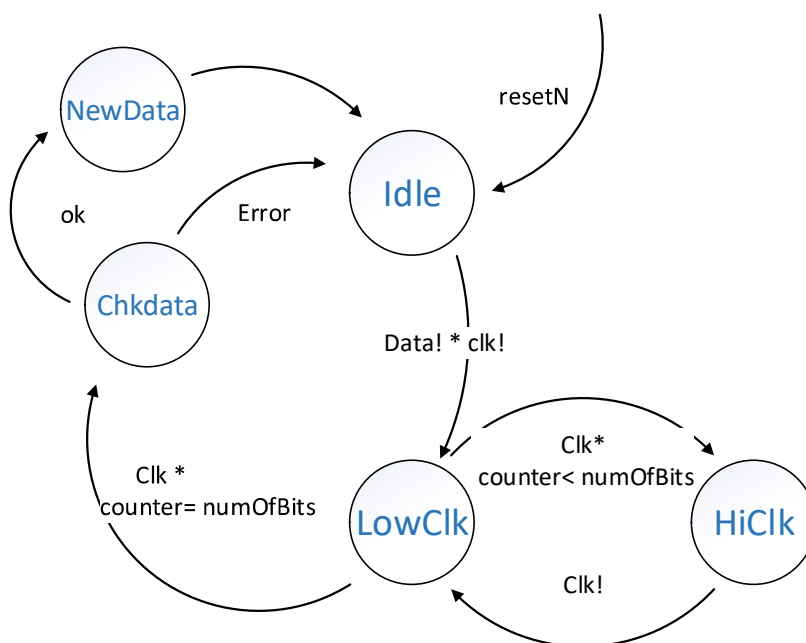
המערכת כוללת גם מונה ברוחב ארבע סיביות שסופר את מספר הסיביות שאוחסנו ברגיסטר ההזזה. היציאות של המונה מזינות את מכונת המצבים.

ליבה של המערכת היא כמובן מכונת מצבים (מסוג Moore) שמשמשת כבקר של היחידה. דיאגרמת המצבים הבאה מתארת את התנהגותה של מערכת זו.

מכונה זו עובדת על שעון 50MHz (שהוא מהיר בהרבה משעון המקלדת).

מכונת המצבים מוזנת גם מאותות הכניסה PS2_DAT ו- PS2_CLK ומפיקה את אות יציאה - .dout_new

בתרגיל ההכנה עליכם לקבוע מהו NUM_OF_BITS, כאשר יודעים את גודלו של רגיסטר ההזזה. כמו כן יודעים שסיביות הזוגיות נבדקת על סמך 9 ביטים : 8 סיביות של קוד המקש וסיבית הזוגיות, ללא התחשבות בביטים הידועים של התחלה (Start) וסיום (Stop) המילה.



3.3.3 תכן יחידת ה - BYTEREC

היחידה שמטפלת בתשדורת ברמת ה - byte, קולטת כמה - bits מהיחידה הקודמת BITREC ומפרשת מצירופים אלו, מהו המקש שנלחץ או נעזב. כדי להדגים זאת, נשתמש למשל בדוגמה של מקשי החצים (Arrows) במקלדת כפי שהוסבר החומר הרקע.

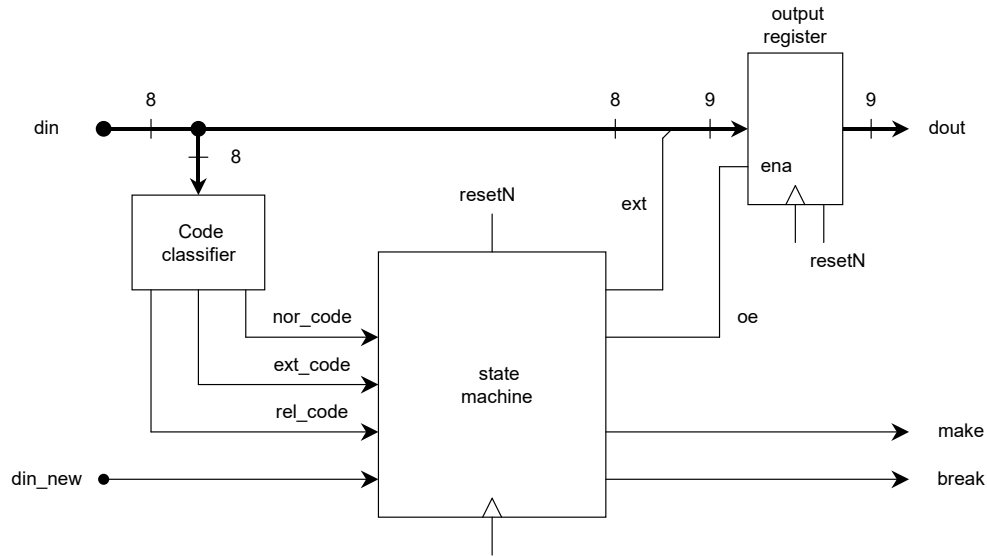
בפרק הקודם תארנו את אופן פעולתם של מקשי המקלדת (הרגילים והמורחבים) ברמת ה - Byte. כל הקודים שתיארנו היו מבוססים על ערך של Byte בסיסי בודד (כדוגמת הערך 72).

הקוד שמתאר את יחידת ה-BYTEREC הוכן עבורך והוא כולל קובץ שמתאר את המודול byterec sv (בשפת System Verilog).

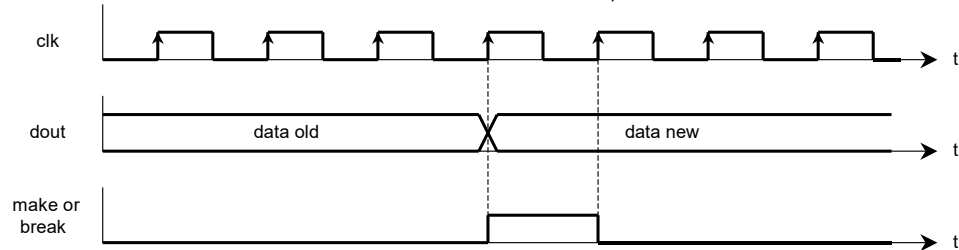
להלן הסבר קצר (תיעוד קצר) על פעולת רכיב ה - byterec שהוכן עבורך.

התכן הפנימי כולל מערכת צירופית לזיהוי סוג הקוד הנכנס (Code Classifier), מכונת מצבים, ורגיסטר יציאה.

תפקיד ה - Code Classifier לזהות האם הקוד הוא רגיל (כמו 72 בדוגמה שלנו) קידומת של קוד מורחב (E0) או קידומת של קוד עזיבה (F0). סוגי היציאה מופקים משלושת היציאות: nor_code, rel_code ו ext_code בהתאמה. רגיסטר היציאה הוא ברוחב 9 סיביות ומורכב משמנה סיביות של אות הכניסה וסיבית גבוהה של אות ext (שהיא "1" לוגי כאשר מדובר בקוד מורחב). האפשרות של הרגיסטר נעשה באמצעות אות oe.



אות היציאה של יחידה זו נראה כך :



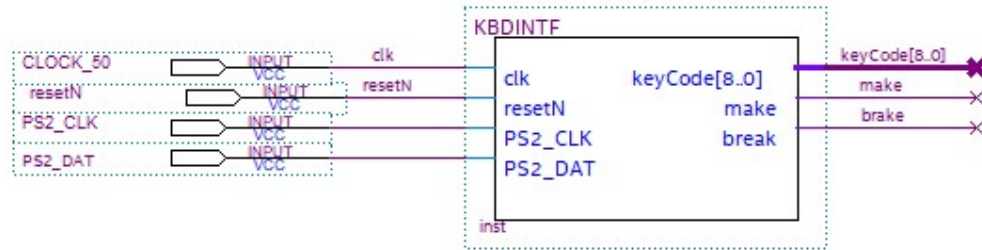
שים לב שהאותות make ו/או break פעילים במשך מחזור שעות אחד (של המערכת 50 MHz) מיד לאחר שהאות dout עודכן במידע חדש. האות make יכול להופיע מספר פעמים, כל עוד המקש המסוים לחוץ.

>> אות היציאה מהיחידה הזאת הוא של 9 ביטים : 8 ביטים של קוד המקש ; אם הביט התשיעי שווה "1" זהו מקש מסוג EXTENDED (E0) <<

3.3.4 ממשק המקלדת הסופי

KBDINTF – הממשק למקלדת בצורתו הסופית (אשר תשלימו במהלך המעבדה DEBUG) מכיל את שלוש היחידות הנ"ל ובעל הכניסות (הלא טריוויאליות) והיציאות הבאות :

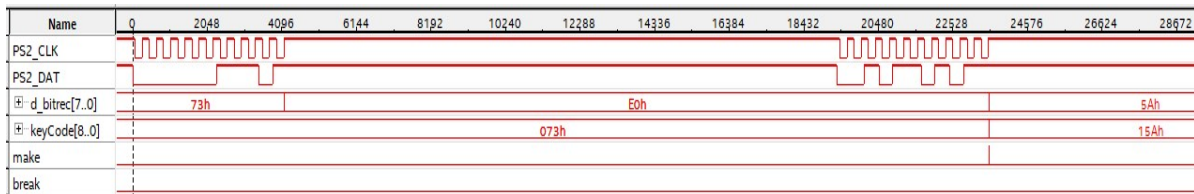
input logic PS2_CLK	שעון המקלדת
input logic PS2_DAT	מידע המקלדת
output logic [8:0] keyCode[8:0]	יציאה של 9 ביט, הבנויה כך :
keyCode[7:0]	- 8 ביט קוד המקש שנלחץ או נעזב
keyCode[8]	- 1 ביט חיווי לסוג המקש : 0 רגיל או 1 Extended
output logic make	חיווי שמקש נלחץ (יתכן מספר פעמים עבור לחיצה ארוכה)
output logic brake	חיווי שמקש נעזב



להלן כמה דוגמאות למקשים ואיך האותות השונים נראים בהקלטות הנתח הלוגי:

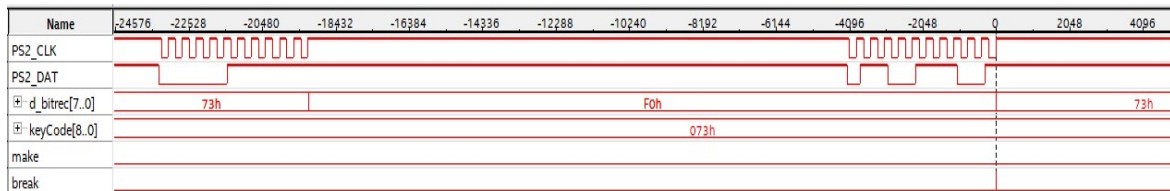
(1) מקש Enter נלחץ:

- $d_bitrec[7..0]$ - הקוד הטורי שמגיע הוא: קודם E0 (מקש מסוג Extended) אחר כך 5A (קוד המקש) שנראים ביציאה מיחידת ה-Bitrec באות $d_bitrec[7..0]$.
- $keyCode[8..0]$ - ביציאה מממשק המקלדת רואים את המספר 5A – 15A הוא קוד המקש ו-1 כי המקש מסוג Extended.
- האות make - הוא 1 לוגי למשך מחזור שען (מערכת) אחד אחרי סיום העברת הביט האחרון (חיווי כי המקש נלחץ).
- האות break - אינו משתנה, כי במקרה זה הקלטת הנתח הלוגי מכוונת לארוע הלחיצה על המקש וההקלטה לא מספיק ארוכה כדי לקלוט את ארוע העזיבה של המקש.

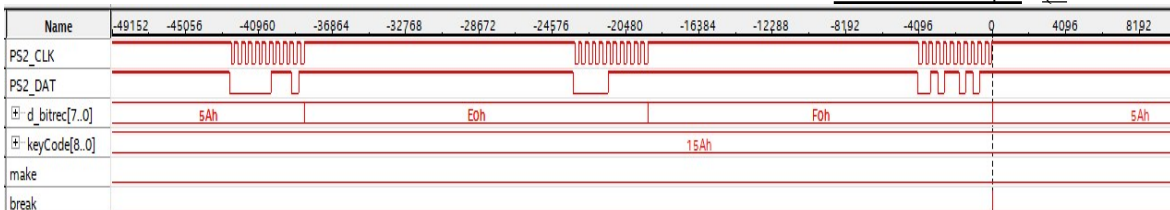


(2) מקש 5 נעזב:

- $d_bitrec[7..0]$ - הקוד הטורי שמגיע הוא: קודם F0 (מקש נעזב) אחר כך 73 (קוד המקש) שנראים ביציאה מיחידת ה-Bitrec באות $d_bitrec[7..0]$.
- $keyCode[8..0]$ - ביציאה מממשק המקלדת רואים את המספר 073 – 73 הוא קוד המקש ו-0 כי המקש רגיל.
- האות make - אינו משתנה, כי במקרה זה הקלטת הנתח הלוגי מכוונת לארוע העזיבה של המקש.
- האות break - הוא 1 לוגי למשך מחזור שען (מערכת) אחד אחרי סיום העברת הביט האחרון (חיווי כי המקש נעזב).



(3) מקש Enter נעזב:



4 מערכת הניסוי לבדיקת החומרה HW_DEBUG עם מכונת RANDOM

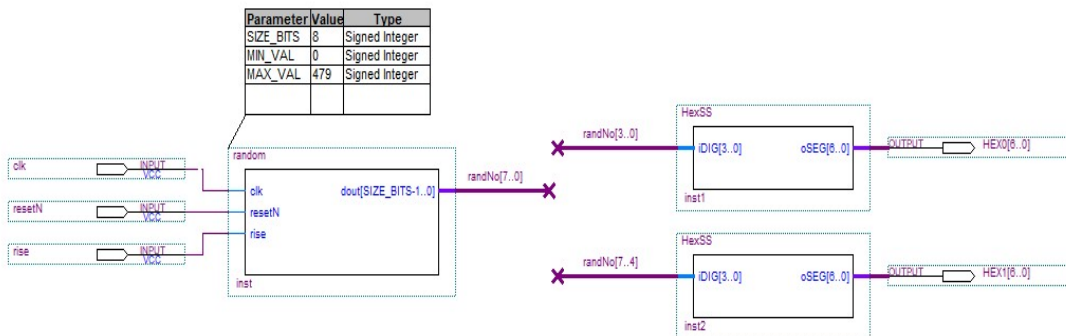
מטרה: במהלך המעבדה תשתמש ביישום נתון לבדיקת מערכות בחומרה. יישום זה בנוי כמכונה ליצירת מספר אקראי RANDOM, כמוסבר להלן.

רכיב ה- **random.sv** כולל מונה בינארי, עם יציאה ברוחב שמונה סיביות, **dout**. זהו מונה שסופר כלפי מטה ובכל פעם שהוא מגיע לאפס (הערך המינימלי) המונה מבצע טעינה של הערך המקסימלי (יכול להיות עד 255) וסופר שוב כלפי מטה, בצורה מחזורית. כאשר אות הכניסה **rise** עולה מ- "0" ל- "1" לוגי ערך המונה שהיה באותו רגע מוצג ביציאה **dout**. לרכיב גם הכניסות הרגילות של איפוס אסינכרוני, **resetN**, וכניסת שעון שמוזנת מהשעון הראשי של לוח ה- DE10 (שפועל בתדר של 50 MHz).

אות הכניסה **rise** מחובר ללחצן KEY1, כך שבסופו של דבר, בכל פעם שלוחצים על KEY1 המערכת מפיקה מספר אקראי כל שהוא ביציאה **dout [7..0]**.

שמונה הסיביות של יציאת המונה מוצגות על גבי 2 ספרות 7segments.

האיור הבא מתאר את המערכת הנתונה לך, **TOP_RANDOM.bdf**:



לסיכום, היציאה **randNo [7..0]** מפיקה מספר אקראי אחר כל פעם שהמשתמש לוחץ על לחצן KEY1.

5 נספח – העשרה – עריכת תוכן של זיכרון וקבועים בזמן אמת ISMCE

בחלקים הקודמים הצגנו את ה - SignalTap ואת יכולות הדיבוג החזקות שלו. כלי דיבוג חזק נוסף שבו משתמשים בדרך כלל כאשר מדבגים מערכת חמרה ושלעתים קרובות הוא גם משלים את היכולות של ה - Signal Tap הוא ה - In System Contents Memory Editor או בקצור ISMCE. כלי זה מאפשר לבצע את הפעולות הבאות:

- ☐ להתבונן בזמן אמת בתוכן של זיכרונות וקבועים במערכת החמרה
- ☐ לשנות בזמן אמת תוכן של זיכרונות וקבועים במערכת החמרה

כל הפעולות הנ"ל נעשות בנוחות רבה באמצעות Quartus ומבלי שיהיה צורך לבצע קומפילציה מחודשת של הפרויקט.

כשם שאין צורך במכשור חיצוני יקר על מנת להפעיל את ה - SignalTap, גם ה - ISMCE אינו דורש מכשור וחמרה חיצוניים. לצורך הפעלתו משתמשים ב - Quartus שיוצר קשר עם משאבי החמרה הפנימיים ברכיבי המתוכנתת באמצעות שרשרת ה - JTAG.

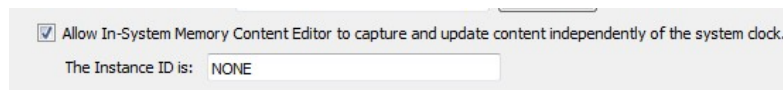
הרכיבים הבאים תומכים באופציה זו:

- ☐ קבוע - LPM_CONSTANT (נראה בניסוי)
- ☐ רכיב זיכרון ROM בעל Port בודד - כמו LPM_ROM
- ☐ רכיב זיכרון RAM בעל Port בודד - כמו LPM_RAM

את יכולת ה - ISMCE ניתן להוסיף לרכיבים מהסוגים הנ"ל באמצעות ה - MegaWizard או באופן ישיר באמצעות שימוש בפרמטר המיוחד שנקרא lpm_hint. תפקידו של פרמטר זה הוא להעביר ל - Quartus כמה מחרוזות תווים שמשמשים כפרמטרים מיוחדים. הטבלה הבאה שמתארת את הפרמטרים המיוחדים הללו, שמועברים כמחרוזות תווים לרכיב תחת lpm_hint:

שם הפרמטר	משמעות הפרמטר
ENABLE_RUNTIME_MOD	אפשר שינוי ערך הזיכרון על ידי Quartus
INSTANCE_NAME	השם של הזיכרון הניתן לשינוי בתוך Quartus

בכדי ש - ISMCE יופעל על רכיב ספציפי, יש לסמן את האפשרות הזו באשף של הרכיב, בזמן יצירת הרכיב או בזמן מאוחר יותר ע"י לחיצה כפולה עליו. הסברים נוספים ב - Quartus Cook Book.



נספח העשרה - עורך הזכרון ISMCE

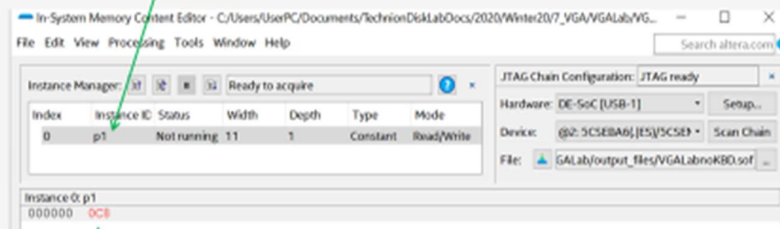
קריאה ושינוי תוכן זיכרון באמצעות ISMCE (In-System Memory Content Editor)

✓ Allow In-System Memory Content Editor to capture and update content independently of the system dock.

מתבצעים באמצעות רכיב lpm_constant שמאפשר גישה לזכרון

שם הזיכרון אותו קוראים/משנים

יש להגדיר כרטיס וקובץ לצריבה ולצירוב לכרטיס :



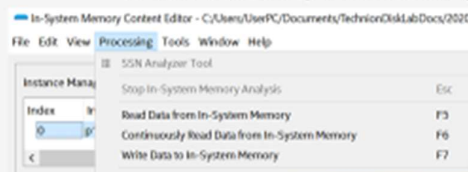
ואז קריאה וכתיבה לזיכרון מתבצעות ע"י התפריט:

לפני קריאה וכתיבה לזיכרון: ??

קריאה של תוכן שונה מקודם: FF

קריאה של תוכן זהה לקודם: FF

מידע חדש שעדיין לא בזיכרון: FF



6 נספח – מימוש זיהוי ה- ENTER

```
always_ff @(posedge clk or negedge resetN)
. . .
if (key_Pressed == KEY_ENTER ) begin
    if (make == 1'b1) enter_pressed <= 1'b1 ;
    if (brakee == 1'b1) enter_pressed <= 1'b0 ;
end

enter_pressed_d <= enter_pressed ; // delay of one clock
. . .
end // always_ff

assign ball_toggle_edge = (( enter_pressed_d == 1'b0 ) && (
enter_pressed == 1'b1 )) ? 1'b1 : 1'b0 ; // rising edge
```

7 נספח – טבלת קודים של כל המקשים (102)

KEY	CODE	KEY	CODE	KEY	CODE
A	01C	'	00E	DELETE	171
B	032	-	04E	END	169
C	021	=	055	PG DN	17A
D	023	\	05D	UP	175
E	024	BKSP	066	LEFT	16B
F	02B	SPACE	029	DOWN	172
G	034	TAB	00D	RIGHT	174
H	033	CAPS	058	NUM	077
I	043	L SHFT	012	KP /	14A
J	03B	L CTRL	014	KP *	07C
K	042	L GUI	11F	KP -	07B
L	04B	L ALT	011	KP +	079
M	03A	R SHFT	059	KP EN	15A
N	031	R CTRL	114	KP .	071
O	044	R GUI	127	KP 0(up)	070
P	04D	R ALT	111	KP 1	069
Q	015	APPS	12F	KP 2(dn)	072
R	02D	ENTER	05A	KP 3	07A
S	01B	ESC	076	KP 4(left)	06B
T	02C	F1	005	KP 5	073
U	03C	F2	006	KP 6(right)	074
V	02A	F3	004	KP 7	06C
W	01D	F4	00C	KP 8	075
X	022	F5	003	KP 9	07D
Y	035	F6	00B	]	05B
Z	01A	F7	083	;	04C
0	045	F8	00A	'	052
1	016	F9	001	,	041
2	01E	F10	009	.	049
3	026	F11	078	/	04A
4	025	F12	007		
5	02E	SCROLL	07E		
6	036	[	054		
7	03D	INSERT	170		
8	03E	HOME	16C		
9	046	PG UP	17D		

8 נספח – טבלת קודים של המקשים כולל MAKE/BREAK

A.2 Keycode tables

KEY	MAKE	BREAK	KEY	MAKE	BREAK
A	1C	F0,1C	R ALT	E0,11	E0,F0,11
B	32	F0,32	APPS	E0,2F	E0,F0,2F
C	21	F0,21	ENTER	5A	F0,5A
D	23	F0,23	ESC	76	F0,76
E	24	F0,24	F1	05	F0,05
F	2B	F0,2B	F2	06	F0,06
G	34	F0,34	F3	04	F0,04
H	33	F0,33	F4	0C	F0,0C
I	43	F0,43	F5	03	F0,03
J	3B	F0,3B	F6	0B	F0,0B
K	42	F0,42	F7	83	F0,83
L	4B	F0,4B	F8	0A	F0,0A
M	3A	F0,3A	F9	01	F0,01
N	31	F0,31	F10	09	F0,09
O	44	F0,44	F11	78	F0,78
P	4D	F0,4D	F12	07	F0,07
Q	15	F0,15	SCROLL	7E	F0,7E
R	2D	F0,2D	I	54	F0,54
S	1B	F0,1B	INSERT	E0,70	E0,F0,70
T	2C	F0,2C	HOME	E0,6C	E0,F0,6C
U	3C	F0,3C	PG UP	E0,7D	E0,F0,7D
V	2A	F0,2A	DELETE	E0,71	E0,F0,71
W	1D	F0,1D	END	E0,69	E0,F0,69
X	22	F0,22	PG DN	E0,7A	E0,F0,7A
Y	35	F0,35	UP	E0,75	E0,F0,75
Z	1A	F0,1A	LEFT	E0,6B	E0,F0,6B
0	45	F0,45	DOWN	E0,72	E0,F0,72
1	16	F0,16	RIGHT	E0,74	E0,F0,74
2	1E	F0,1E	NUM	77	F0,77
3	26	F0,26	KP /	E0,4A	E0,F0,4A
4	25	F0,25	KP *	7C	F0,7C
5	2E	F0,2E	KP -	7B	F0,7B
6	36	F0,36	KP +	79	F0,79
7	3D	F0,3D	KP EN	E0,5A	E0,F0,5A
8	3E	F0,3E	KP .	71	F0,71
9	46	F0,46	KP 0	70	F0,70
'	0E	F0,0E	KP 1	69	F0,69
-	4E	F0,4E	KP 2	72	F0,72
=	55	F0,55	KP 3	7A	F0,7A
\	5D	F0,5D	KP 4	6B	F0,6B
BKSP	66	F0,66	KP 5	73	F0,73
SPACE	29	F0,29	KP 6	74	F0,74
TAB	0D	F0,0D	KP 7	6C	F0,6C
CAPS	58	F0,58	KP 8	75	F0,75
L SHFT	12	F0,12	KP 9	7D	F0,7D
L CTRL	14	F0,14	]	5B	F0,5B
L GUI	E0,1F	E0,F0,1F	:	4C	F0,4C
L ALT	11	F0,11	'	52	F0,52
R SHFT	59	F0,59	,	41	F0,41
R CTRL	E0,14	E0,F0,14	.	49	F0,49
R GUI	E0,27	E0,F0,27	/	4A	F0,4A